

T-verse Architecture: AI Photo-Digitizing & Marker Generation

Project: T-verse ERP Supply Chain Module

Brand Context: Thalasi® Production Line

Core Tech Stack: Java 25, Spring Boot 3.4, React, Python FastAPI, OpenCV

1. Executive Summary

This document outlines the technical architecture for the AI-Assisted Photo-Digitizing and Algorithmic Marker Generator. This module replaces manual pattern tracing by utilizing computer vision to extract 2D vectors from physical patterns using a calibration marker. It then applies heuristic bin-packing algorithms to nest these patterns efficiently, generating both a digital PDF for plotter printers and an interactive visual guide for warehouse cutting masters.

2. Microservices Architecture

The system is intentionally decoupled to separate the heavy machine learning matrix calculations from the high-concurrency transactional ERP environment.

- **Frontend (React):** Captures the image using the device camera, ensuring the device is level. Uploads the raw image to a storage bucket and passes the URL to the backend.
- **Orchestrator (Java 25 & Spring Boot 3.4):** Acts as the traffic controller. Receives the request, manages the database transactions, and makes asynchronous HTTP POST calls to the vision microservice. Utilizing Virtual Threads prevents Tomcat thread exhaustion during high-volume processing.
- **Vision Engine (Python FastAPI & OpenCV):** Ingests the image URL, performs edge detection, scales the polygon based on the known calibration square dimensions, and returns a JSON payload of 2D vector coordinates.

3. Integration & API Contract

The following defines the communication protocol between the Spring Boot orchestrator and the Python vision service.

Request Payload (Java -> Python)

```
POST /v1/vision/extract-pattern
{
  "pattern_id": "PTN-THALASI-OVERSZ-M-FRONT",
  "image_url": "https://storage.tverse.local/raw-scans/scan_8839.jpg",
  "calibration_square_cm": 10.0
}
```

Response Payload (Python -> Java)

```
{
  "status": "success",
  "pattern_id": "PTN-THALASI-OVERSZ-M-FRONT",
  "real_world_dimensions_cm": {
    "width": 55.4,
    "height": 72.1
  },
  "vector_coordinates": [
    {"x": 0.0, "y": 0.0},
    {"x": 55.4, "y": 0.0},
    {"x": 55.4, "y": 40.2},
    {"x": 40.1, "y": 72.1},
    {"x": 15.3, "y": 72.1},
    {"x": 0.0, "y": 40.2}
  ],
  "confidence_score": 0.98
}
```

4. Algorithmic Nesting & Output

Once the vectors are stored in the T-verse database, the system batches production orders and runs a 2D irregular bin-packing algorithm to optimize fabric yield. The final outputs are:

- **Plotter Integration:** The Java backend dynamically generates a continuous, 1:1 scale wide-format PDF marker for industrial cutting printers (utilizing libraries like Apache PDFBox).
- **Tablet UI:** The React frontend renders an interactive HTML5 <canvas> map. This high-contrast interface provides color-coded sizing, explicit ply-count calculations, and grainline alignment checks for the cutting master.

5. Strategic Impact

Implementing this architecture demonstrates advanced enterprise design patterns. Decoupling the Python CV layer from the Spring Boot core showcases an understanding of system scalability, appropriate technology selection, and real-world cost optimization (reducing fabric waste). This robust separation of concerns serves as an excellent technical showcase for system design evaluations during the upcoming August placements.